

COS214 Practical Assignment 2

Caleb Jennings - u25173805

Anchen Kruger - u25073703

Chloe Larsen - u25004141

Task 1

1.1)

Our event is **The Heap & Heap Artisanal Bazaar**, which is a weekend long open-air multivendor marketplace that celebrates the local artisans, musicians, craftspeople, and food producers. The event takes place in a park in Pretoria where the market is split up into different precincts where each precinct specialises in different categories, zones, of the artisan experience.

At this market, operational resources are allocated on demand and safely managed across hierarchical operational zones. The market coordinates a bustling ecosystem of culinary stalls, express drink kiosks, seating facilities, venues, workshops, security gates, and mobile cleanup crews across multi-tier precincts. Operational commands and emergency alerts originate from a central event control node and cascade down through nested subsections to keep the entire market synchronized without memory leaks or race conditions.

What is offered at the market:

- Artisans sell handmade goods across multiple precincts (pottery, textiles, jewelry, woodwork, etc.)
- Musical performances happening on various stages
- Information points help visitors navigate and find specific artisans
- Medical personnel that ensure visitor safety
- Food stalls for visitors to travel around the world through the available foods

Concrete Leaf Types:

Get their own subclasses.

- **VendorBooth** - An artisan selling goods; responds to weather alerts by securing displays, capacity alerts by limiting entry
- **PerformanceStage** - Hosts scheduled performances; responds to weather by pausing shows
- **InformationDesk** - Visitor information point; remains operational during most alerts
- **FirstAidPoint** - Medical assistance; always operational regardless of other alerts
- **FoodStation** - Food preparation and tasting point; reacts to stock alerts by closing, capacity alerts by limiting service

Composite Grouping Types:

Don't get their own subclasses. Specified in the name of the EventGroup instance.

- **OperationalSector** - Non-public operational grouping
- **Venue** - Enclosed or semi-enclosed area
- **Precinct** - Major themed area
- **Zone** - Sub-area within a precinct

1.2)

Composite:

Component	EventComponent	Abstract interface for all event components
Leaf	EventUnit, VendorBooth, PerformanceStage, FoodStation, InformationDesk, FirstAidPoint	Concrete operational units with distinct behaviors
Composite	EventGroup	Contains children (EventComponent*), delegates operations recursively

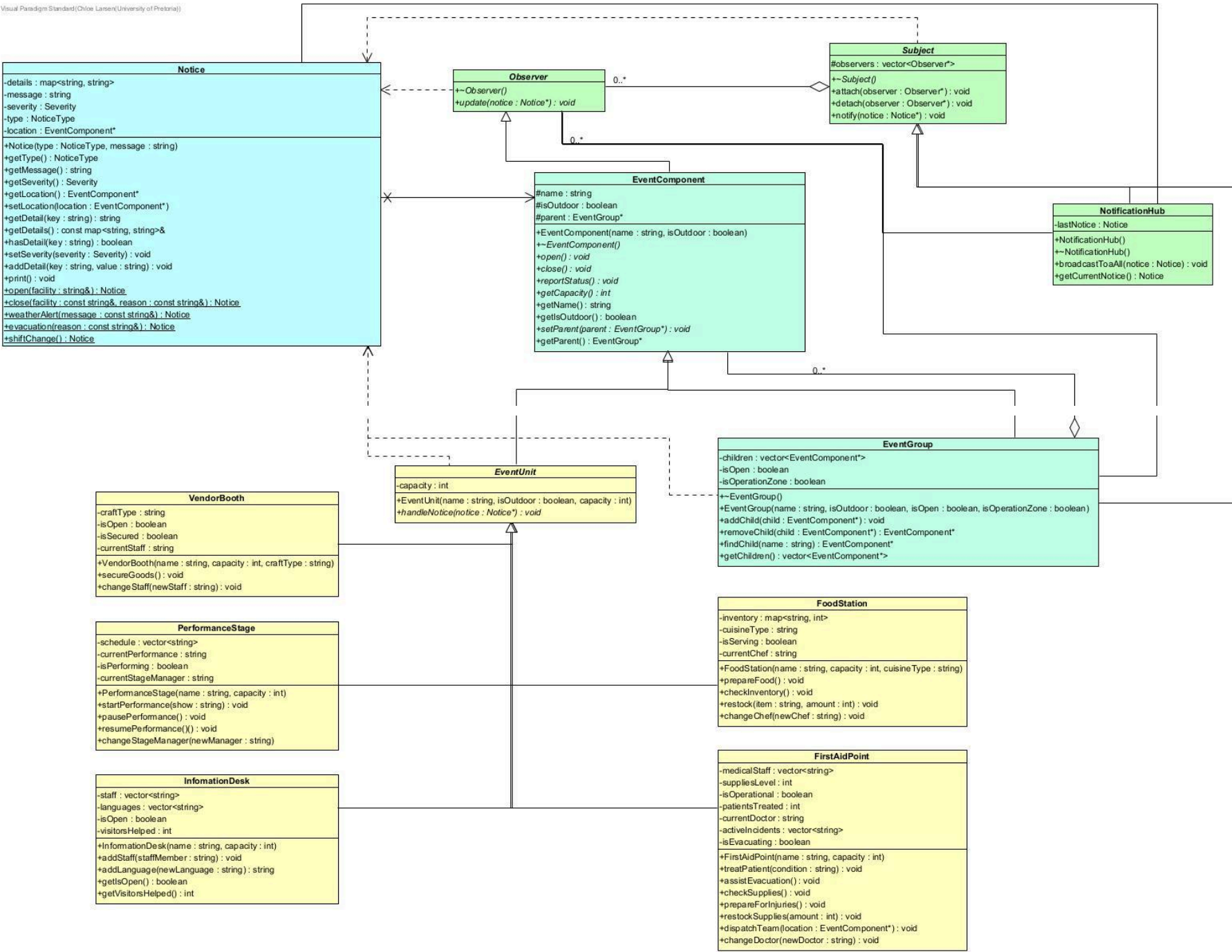
Observer:

Subject	Subject	Interface for attaching/detaching/notifying observers
ConcreteSubject	NotificationHub	Central broadcast hub that disseminates event-wide notices.
ConcreteSubject	EventGroup	Also acts as subject for its children (cascades notifications)
Observer	Observer	Interface for receiving updates
ConcreteObserver	EventGroup	Forwards notifications to children
ConcreteObserver	EventUnit	React to notices with polymorphic behavior

Multi-Role Classes:

- EventGroup
 - **Composite role:** Contains and manages children
 - **ConcreteObserver role:** Receives notifications from parent/NotificationHub
 - **ConcreteSubject role:** Notifies child observers
- VendorBooth, PerformanceStage, FoodStation, WorkshopTent, InformationDesk, FirstAidPoint, SecurityCheckpoint
 - **Leaf role:** Terminal nodes in the composite tree
 - **ConcreteObserver role:** Receive and react to notifications

1.3)



1.4)

- (a) The composite pattern is generally implementation of a part-whole tree because:
- Both EventGroup (Composite) and EventUnit (Leaf) implement the same EventComponent interface with *open()*, *close()*, *reportStatus()*, and *getCapacity()*.
 - EventGroup contains a `std::vector<EventComponent*>` of children, which can be either Leaf or Composite objects. This allows unlimited nesting depth.
 - All composite operations recursively delegate to children:
 - *open()* → opens all children
 - *close()* → closes all children
 - *reportStatus()* → reports status of all children
 - *getCapacity()* → aggregates capacity of all children
 - Clients can treat all components uniformly without knowing if they are working with a Leaf or a Composite.

Therefore the composite is a part-whole tree and the uml design follows the composite design rules.

- (b) The Observer pattern is essential to the design as:
- Not all units need to hear all notices. Units can be attached or detached at runtime based on operational requirements.
 - EventGroup acts as both Observer (receiving from parent) and Subject (forwarding to children), enabling notifications to flow down the composite tree.
 - Different observers react differently to the same notice through the *handleNotice()* method, without requiring type checking in the Subject.
 - The NotificationHub and EventGroup subjects do not need to know about every concrete leaf type. Observers register themselves through the *attach()* method.
 - A single notice (e.g., WeatherAlert) must reach many different units simultaneously. The Observer pattern handles this naturally.
 - New leaf types can be added without modifying any existing Subject code. They simply implement Observer and register.

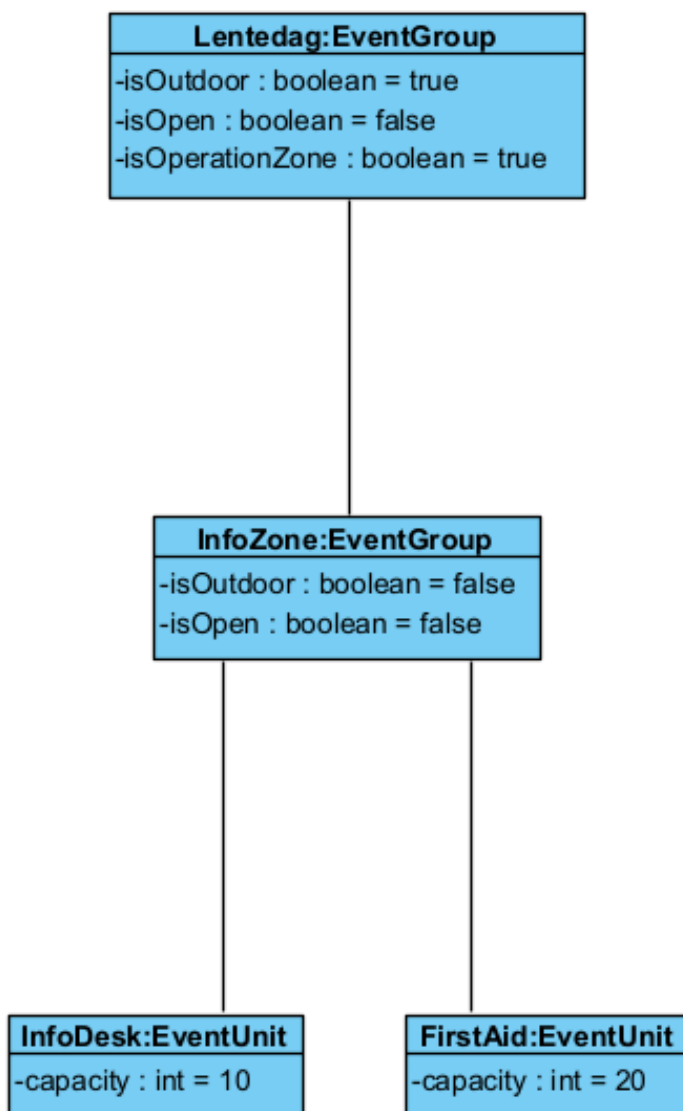
Therefore the use of the Observer pattern helps to ensure that the notifications can be viewed and different implementations of the notifications can be implemented for the different leaf types in the composite pattern

- (c) For the event components:
- EventGroup owns its children this means that each EventGroup contains a `std::vector<EventComponent*>` of its children. The EventGroup is responsible for deleting these children in its destructor.
 - Every single event component contains a parent variable that Each EventComponent has exactly one parent (except the root, which has parent = nullptr), this pointer is a non-owning back-pointer used for tree navigation. It is not used for ownership or deletion.
 - The client owns the root element and is responsible for its deletion, which cascades to all children.
 - When *removeChild()* is called, ownership transfers to the caller, who must then delete or re-add the component.
- (d) For the observers:
- Both Subject (and its subclasses NotificationHub and EventGroup) store observer references as raw pointers (*Observer**). These are non-owning references. The use of non-owning references ensures that there is no double deletion and maintains a clear single ownership hierarchy.
 - All observers (leaves and composites) are owned by the Composite tree. The Subject does NOT delete observers.

- Observers remain alive while registered and detach themselves before destruction.
 - When an observer is destroyed, it calls *detach()* on all Subjects it is registered with to prevent dangling pointers.
- (e) EventGroup plays three roles simultaneously, which is not pattern misuse as
- There is different collaborations that takes place at different levels:
 - As Observer: Receives notifications from parent or NotificationHub (above)
 - As Subject: Forwards notifications to children (below)
 - The observer participant and the subject participant have different purposes:
 - The observer role: Needs to know about external changes that are affecting the subject
 - The subject role: Must tell its children about the changes it is aware of.
 - Single Responsibility Principle Still Holds: Each role has a clear, distinct purpose:
 - Composite: Managing the part-whole hierarchy
 - Observer (receiving): Getting external notifications
 - Subject (sending): Distributing notifications to children

Task 2

2.3)



2.4) The entire tree (root and subtree) are correctly released exactly once due to recursive destruction. When a group (composite) is deleted, it recursively deletes all its children. Some of these children can themselves be groups too. This way, every component is removed. When a component is transferred between composites, its parent - and thus its owner - changes. This is done by first removing it from its current parent and then adding it to its new parent. When the subtree it belongs to is deleted, its new parent will clean it up.

Task 3

3.2) Observers themselves do not detach themselves from any subjects automatically. However, EventComponents, being Observers, do detach themselves from their parent node automatically in their destructor, and when their parent is changed. This means that all non-root nodes will automatically detach themselves when necessary. As for the root, because it is the only node that attaches to the NotificationHub (a concrete subject), it is manually detached at the end of the program.

3.5) We chose to use a push implementation. A pull implementation would have been better to use for subjects with complicated states that are distributed through multiple fields in the subject. However, the subject in this practical (NotificationHub) does not have any state. This means that all information used in event listeners must be sent during the notification itself. We do this using our *Notice* class which contains detailed information about the event which observers can use to handle it appropriately.

Task 4

4.1) Event Rules:

- **Open** - Causes units to open/activate and is used by all EventComponents
- **Close** - Causes units to close/deactivate and is used by all EventComponents
- **Evacuate** - Triggers emergency evacuation procedures and is used by all EventComponents - most places close down and the staff at first aid point assist in getting everyone evacuated
- **MedicalAlert** - Causes most EventComponents to close and makes FirstAidPoint move to the location where the alert takes place if it is operational and has enough staff
- **ScheduleChange** - Swap out current staff working at different locations such as the person providing information at the info desk, the chef at a food stall, or the doctors on duty at a first aid point

4.4) Original Features:

1.) Shift Change system

A staff rotation system that allows personnel changes across all event units. When a shift change occurs, staff members are updated throughout the event automatically via notifications.

Adding this did not turn object into god objects as each leaf handles its own staff changes independently, no central staff manager controlling everything, the staff details are passed via the notice, not queried from a central authority and polymorphism was used to implement this feature, done the with *handleNotice(Notice *notice)* function.

Code example (`void VendorBooth::handleNotice(Notice *notice)` - lines 42-47):

```

case NoticeType::ScheduleChange:
    if (notice->hasDetail("new_staff"))
    {
        changeStaff(notice->getDetail("new_staff"));
    }
    break;

```

2.) Medical Alert System with Team Dispatching

A comprehensive medical emergency response system that dispatches medical teams based on severity, tracks patients, and manages supplies.

This does not make an object a god object as the medical response logic is contained within FirstAidPoint, each first aid point managed its own staff and supplies and it uses polymorphism through the *handleNotice(Notice *notice)* function.

Code example (`void FirstAidPoint::handleNotice(Notice *notice` - lines 200-220)):

```

case NoticeType::MedicalAlert:
{
    std::string condition = notice->hasDetail("condition") ?
notice->getDetail("condition") : "Unknown condition";
    std::string location = notice->hasDetail("location") ?
notice->getDetail("location") : "Unknown location";

    std::cout << "" << name << " is responding to medical emergency!"
<< std::endl;
    std::cout << "    Condition: " << condition << std::endl;
    std::cout << "    Location: " << location << std::endl;

    if (location != "Unknown location")
    {
        dispatchTeam(location);
    }
    if (condition != "Unknown condition")
    {
        treatPatient(condition);
    }

    checkSupplies();
    break;
}

```

3.) Multiple Language Support Feature

The Information Desk supports multiple languages to assist international visitors at the bazaar. Staff can add new languages dynamically, and the desk tracks which languages are offered.

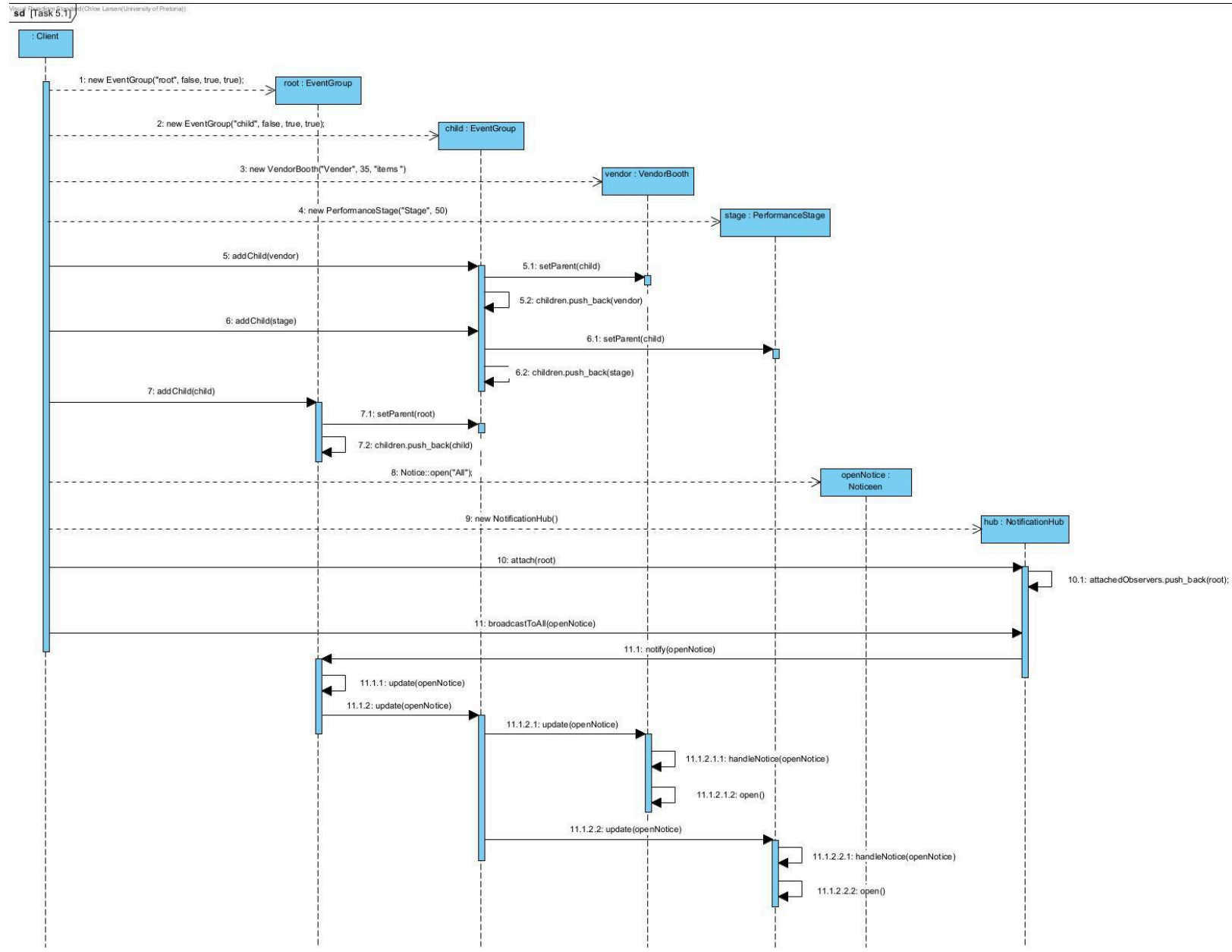
There is no god object created with this feature as Language management is contained within InformationDesk, and each information desk managed their own languages as there is no central control class for the information desks and polymorphism is implemented through the *handleNotice(Notice *notice)* function.

Code example (`std::string InformationDesk::addLanguage(std::string newLanguage)` - lines 14 - 26):

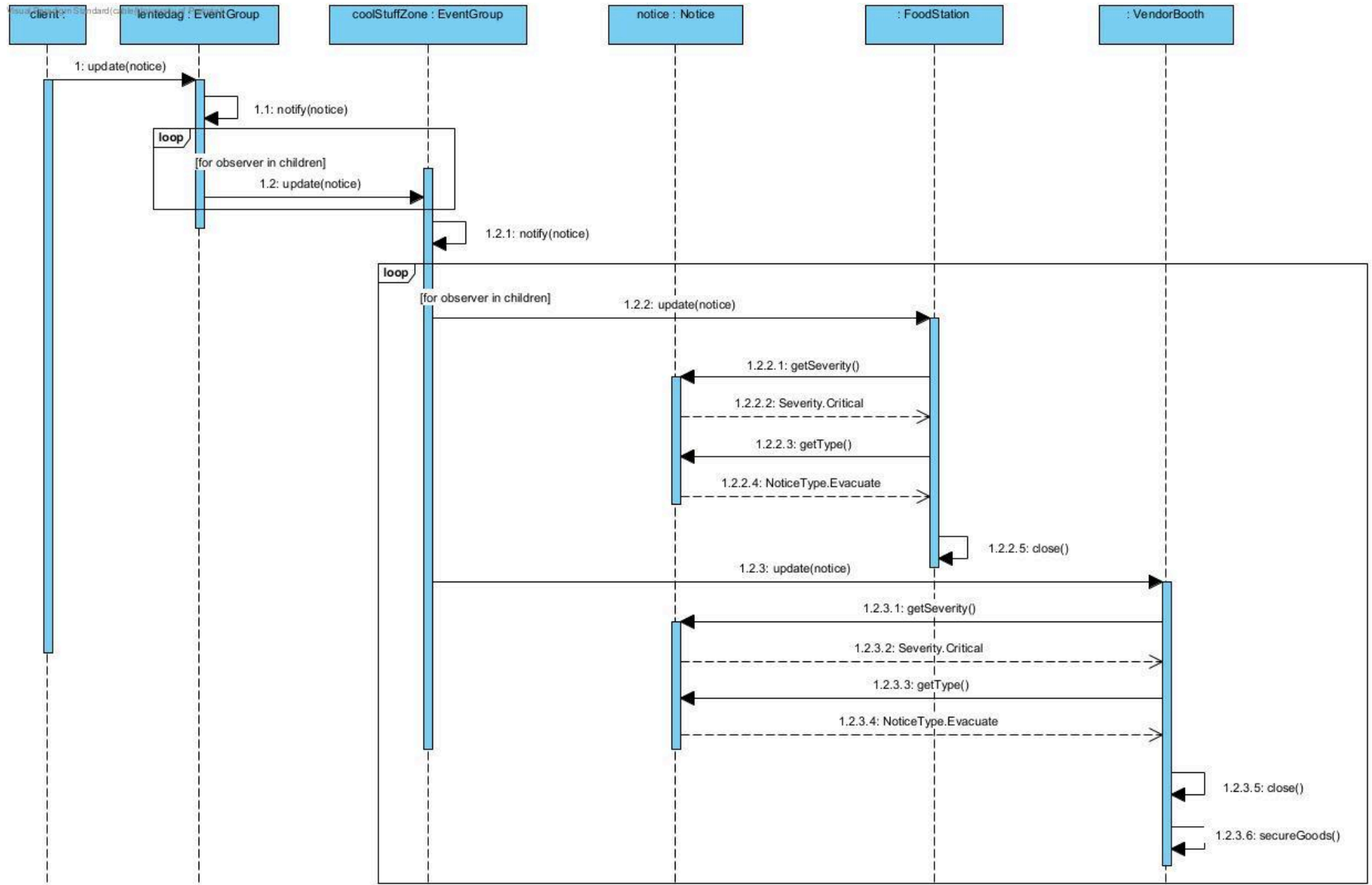
```
{
    for (const auto &lang : languages)
    {
        if (lang == newLanguage)
        {
            std::cout << "Ahh ma cheri, we can already speak " <<
newLanguage << std::endl;
            return newLanguage;
        }
    }
    languages.push_back(newLanguage);
    std::cout << "Oui oui bonjour! Hola, tudo bem? Hallo hoe ganit! We can
speak even more languages now like, " << newLanguage << std::endl;
    return newLanguage;
}
```

Task 5

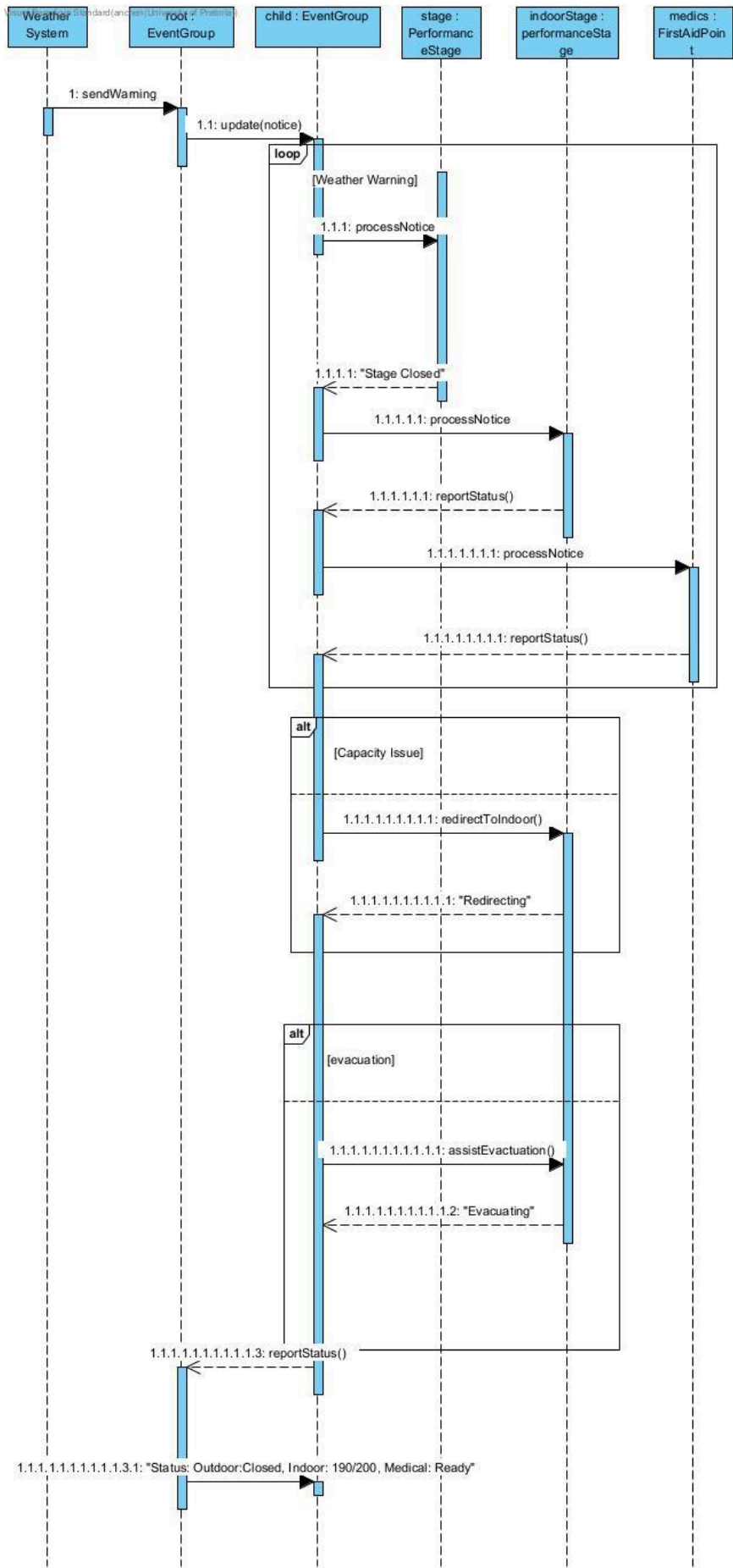
5.1) SD1 Building and registering part of the event



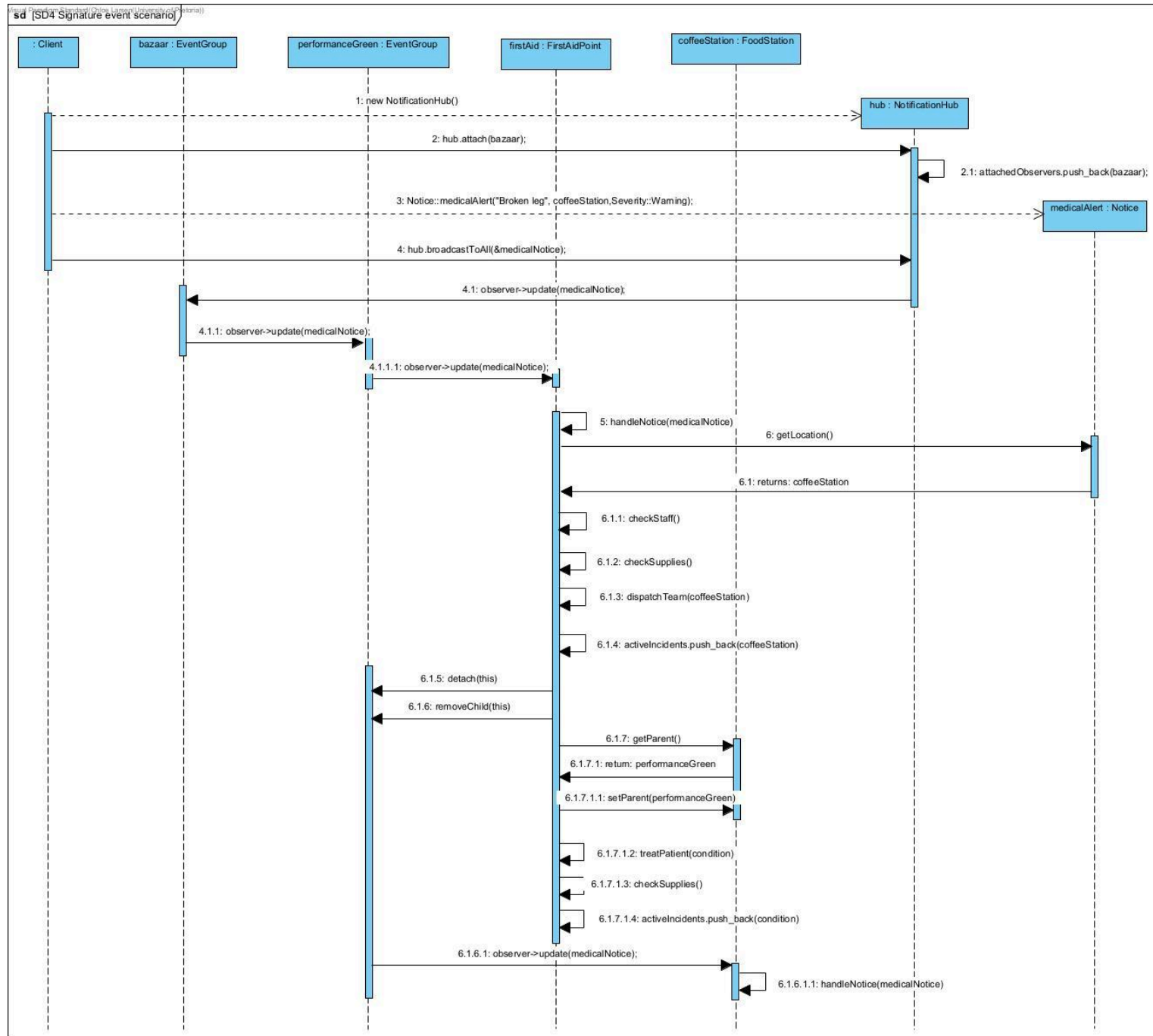
5.2) SD2 Cascading event notification



5.3) SD3 Conditional event response and Composite behaviour



5.4) SD4 Signature event scenario



Task 6



Task 7

7.4)

[GitHub Link](#)

We used GitHub for collaboration as follows: Whenever one of us wanted to work on a new feature, we created a new branch locally, made a few commits with the new changes, and then pushed the branch to the repo. Once on the repo, the person who made those changes opened a pull request for the branch and requested review from one or both of the other members. If the changes were not approved, the pull request was closed and the person making the changes had to revise their additions. Otherwise, the changes are merged into main and the feature branch gets auto closed. We also used

issues to keep track of who still had to do what, and sometimes attached issues to pull requests to automatically close the issue along with the pull request. However, sometimes we pushed straight to main for small changes like additional documentation or README changes.

Branches and pull requests were very helpful because they let us review each other's code before merging it into main, meaning all of our code on the main branch is agreed to be quality code.

We divided the work based on functionality. Chloe created the majority of the UML class diagram, Caleb focused on the observer design pattern, and Anchen focused on the composite design pattern. However, Caleb and Anchen worked together closely as the two design patterns overlapped significantly. Additionally, Chloe implemented the situational event handling. Whoever created a class or method also wrote the documentation (comments) for it, and Anchen later generated the Doxygen documentation. Each member completed a sequence diagram relating to what they focused on, with Anchen completing two.

A good example of collaboration between commits was Chloe creating the structure of the concrete leaves in the composite design pattern, and then Anchen implementing them.

- [Adding h files for composite pattern](#) - Chloe adding header files
- [Did vendorbooth](#) - Anchen completing a concrete leaf
- [Done with InformationDesk](#) - Anchen completing another concrete leaf